Rodrigo Fernandez

Senior AI Full Stack Engineer
Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernández-9a5a0664 | rodrigobfdev@gmail.com

Summary

Senior **AI Full Stack Engineer** with experience building production systems across **healthcare, fintech, SaaS, logistics, and enterprise automation**, using **React 18.x**, **Next.js 13.x–14.x**, **Node.js 18.x–20.x**, **Python 3.10–3.12**, **FastAPI 0.100+**, and **OpenAI/LangChain 0.1.x–0.2.x**. Specialized in AI-powered dashboards, RAG workflows, secure APIs, vector search, cloud deployment, and scalable frontend architecture, with strong focus on reducing latency, improving user workflows, fixing production defects, and migrating legacy platforms into modern AI-ready systems.

Skills

- **Frontend: React 16.x–18.x**, **Next.js 12.x–14.x**, **TypeScript 4.x–5.x**, JavaScript ES6+, Redux Toolkit, Zustand, React Query, Material UI, Tailwind CSS, HTML5, CSS3, Responsive UI
- **Backend: Node.js 14.x–20.x**, Express.js 4.x, NestJS 9.x–10.x, **Python 3.9–3.12**, **FastAPI 0.95+**, Flask 2.x, REST API, GraphQL, WebSocket, Background Jobs
- **AI / LLM: OpenAI API**, **LangChain 0.1.x–0.2.x**, LangGraph, RAG, Prompt Engineering, Function Calling, Embeddings, Vector Search, Document AI, AI Agents, Evaluation Workflows
- **Database:** PostgreSQL 12.x–16.x, MySQL 8.x, MongoDB 5.x–7.x, Redis 6.x–7.x, Pinecone, Weaviate, FAISS, Elasticsearch 7.x–8.x
- **Cloud / DevOps:** AWS, GCP, Docker, Kubernetes, GitHub Actions, GitLab CI/CD, Terraform, Nginx, CloudWatch, S3, Lambda, ECS, Cloud Run
- **Testing / Quality:** Jest, React Testing Library, Cypress, Pytest, Postman, Swagger/OpenAPI, Sentry, Datadog, Prometheus, Grafana

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Led development of an **AI-powered enterprise operations platform** using **React 18.x**, **Next.js 14.x**, **Node.js 20.x**, and **Python 3.12**, helping internal users reduce manual ticket review time by **38%**.
- Designed a secure **RAG workflow** with **LangChain 0.2.x**, OpenAI embeddings, PostgreSQL 16, and vector search to answer customer infrastructure questions from policies, logs, and support documents.
- Migrated legacy React class components from older **React 16.x** patterns into reusable **React 18.x functional components**, improving rendering stability and reducing UI regression issues by **31%**.
- Built FastAPI-based AI services with **Python 3.11–3.12**, async workers, Redis queues, and structured logging to process long-running document analysis without blocking user requests.
- Solved a recurring production bug where AI responses returned outdated policy details by adding source freshness checks, chunk metadata validation, and confidence scoring before final answers.
- Implemented role-based access control across AI dashboards, API endpoints, and document collections so support, security, and operations teams could safely use shared AI tools.
- Improved frontend performance by replacing heavy state logic with **React Query**, memoized table rendering, and pagination, reducing dashboard load time by **42%**.

- Created an AI incident summarization feature that converted logs, alerts, and customer notes into clean investigation summaries with action items and risk labels.
- Refactored Node.js APIs from callback-heavy services into modular **TypeScript 5.x** services, improving maintainability and reducing production error rates by **27%**.
- Integrated **OpenTelemetry**, Prometheus, Grafana, and Sentry to trace AI request failures, slow vector queries, timeout errors, and unstable third-party model responses.
- Partnered with DevOps teams to deploy AI microservices through Docker, Kubernetes, and GitHub Actions, creating safer staging validation before production releases.
- Used **AWS Certified Developer** knowledge to improve cloud deployment security, environment separation, secret handling, and monitoring for AI-enabled customer platforms.

NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed enterprise web applications for fintech, healthcare, and logistics clients using **React 15.x–16.x**, **Angular 2.x–8.x**, **Node.js 6.x–12.x**, **Python 3.6–3.8**, and **Django 1.11–3.0**.
- Built practical **machine learning** and data-processing features with **Python**, scikit-learn, Pandas, TensorFlow 1.x–2.x, and Elasticsearch 5.x–7.x for search and classification use cases.
- Improved internal search relevance by **26%** by tuning Elasticsearch analyzers, weighted fields, synonym rules, and backend ranking logic for business workflows.
- Migrated legacy server-rendered Django pages into API-driven React and Angular modules with reusable UI components and clearer frontend-backend separation.
- Fixed a reporting defect where cached filters showed incorrect totals by redesigning cache keys, correcting SQL joins, and adding regression tests.
- Built secure REST APIs with role-based permissions, JWT authentication, request validation, and structured error handling for client-facing platforms.
- Optimized Node.js service latency by **34%** by improving connection pooling, reducing blocking operations, batching requests, and reviewing slow SQL queries.
- Integrated payment, identity, notification, and document services using secure webhooks, retry handling, signature validation, and failure logging.
- Upgraded frontend build systems with Webpack 3.x–4.x, Babel 6.x–7.x, ESLint, and CI checks to reduce release defects across active client projects.
- Created automated test coverage with Jest, Cypress, Pytest, and Postman collections to reduce frontend and backend regression issues.
- Worked flexibly across frontend, backend, database, ML prototype, and DevOps support tasks depending on client priority and release pressure.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built eCommerce, marketing, and customer portal features using **AngularJS 1.x**, **jQuery 1.x–2.x**, **Node.js 0.12–6.x**, **Express 4.x**, **Django 1.7–1.10**, and **MySQL 5.6–5.7** for high-traffic digital business applications.
- Implemented rule-based **recommendation and personalization** features with **Python 2.7–3.5**, scheduled batch scripts, customer behavior data, and product category scoring before modern LLM tooling became widely adopted.
- Improved checkout completion by **18%** by fixing client-side validation gaps, reducing unnecessary form steps, optimizing mobile layouts, and cleaning slow JavaScript execution paths.
- Migrated legacy **jQuery-heavy** screens into structured **AngularJS 1.x** modules and early **React 0.14–15.x** components where reusable UI patterns improved maintainability.
- Resolved duplicate payment webhook issues by adding **idempotency checks**, transaction status validation, MySQL constraints, and clearer reconciliation logs for support teams.

- Built REST-style backend services with **Express 4.x**, **Django REST Framework 3.x**, **Redis 2.8–3.2**, and MySQL queries for orders, campaigns, inventory, and customer profile workflows.
- Reduced API error rates by **22%** by adding request validation, centralized exception handling, safer deployment smoke tests, and more predictable backend response formats.
- Created business reporting dashboards with **Bootstrap 3**, chart libraries, CSV exports, role-based filters, and scheduled email reports for marketing and operations users.
- Upgraded frontend delivery from manual script management to **Gulp**, **Grunt**, **Browserify**, and **Webpack 1.x** workflows to reduce release mistakes and browser-specific asset issues.
- Worked flexibly across frontend fixes, backend API tickets, database scripts, QA debugging, and production support while keeping delivery realistic for client timelines.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported web application development using **JavaScript**, **jQuery 1.8–1.10**, **Backbone.js 0.9–1.x**, **Python 2.7**, **Django 1.5**, and **MySQL 5.5**.
- Built internal admin screens, search pages, customer forms, and reporting tools while learning production debugging, code reviews, and release discipline.
- Helped improve page response time by **19%** by reducing duplicate SQL queries, compressing static assets, and cleaning inefficient template rendering logic.
- Fixed form submission bugs caused by inconsistent browser behavior by adding validation rules, server-side checks, and clearer user-facing error messages.
- Created basic recommendation and tagging utilities using Python scripts, keyword scoring, and scheduled jobs for content organization workflows.
- Assisted migration from fragile inline JavaScript toward reusable modules, cleaner event handling, and maintainable frontend structure.
- Maintained REST-style endpoints, authentication flows, file upload features, and database-backed reporting screens under senior engineer guidance.
- Added unit and functional tests for high-risk form, authentication, and reporting paths to reduce repeated regression bugs during release cycles.
- Worked across frontend fixes, backend tickets, database scripts, QA support, and deployment troubleshooting to build flexible full stack engineering habits.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal software tooling with Python scripting, SQL queries, and backend debugging while learning enterprise engineering practices, source control, and structured code review.
- Assisted senior engineers with data validation scripts, API testing, and automation tasks that improved repeatable checks across internal development workflows.
- Gained practical experience in production-quality engineering, documentation, test discipline, and communication across distributed teams working on large-scale software systems.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*